Project Report

CAS 782 (2026W)

Vehicle Distributed Online Parameter Estimation and Auto-Park Digital Twin

Author
Samuel Khzym khzyms@mcmaster.ca khzyms

February 15, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster CAS by Dr. Istvan David.

1 Executive summary

This project aims to develop a digital twin for the automotive space which runs in the cloud and which the physical twin (one-tenth scale vehicle) communicates with wirelessly. The digital twin periodically receives vehicle kinematics data continuously from the physical twin during manual driving to update a vehicle dynamics model. This involves vehicle speed, acceleration, yaw rate, and GPS location. This allows for continuous updates to the vehicle model parameters to allow for any deviations in vehicle parameters (mass, tire pressure, etc). It will also evaluate the tolerance of the digital twin to perform well under an unreliable transmission medium (wireless).

Once the physical twin is ready to do an automated parking maneuver, which is generally very compute and memory heavy, it will transmit its information to the digital twin. This involves image and point cloud data from the onboard camera and LiDAR. The digital twin will then determine an optimal trajectory for the physical twin to follow and transmit said trajectory to the physical twin, which will then attempt to follow the provided trajectory.

The intention of the majority of the work is to go towards my Master's thesis, and I would be interested in writing a journal paper on the digital twin system design and fault-tolerant properties of the system.

2 Scope and requirements

2.1 Scope

The scope of the project will be broken up into a physical twin (the vehicle), and the digital twin (cloud-based application). The physical twin shall be implemented in two domains, simulation and in-vehicle.

The first domain for the physicial twin is in simulation, where a virtual vehicle model provided by creators of the RoboRacer framework with variable dynamics parameters. The virtual vehicle model shall represent the physical twin and periodically communicate its state to the digital twin while manually driving. The state of the simulated physical twin will consist of its GPS location, speed, yaw rate, acceleration, and steering wheel angle. The digital twin will periodically re-optimize the vehicle parameters on the most recently received vehicle data. Once the auto-park maneuver starts, the simulated vehicle shall send the camera and LiDAR data to the digital twin, which will find the obstacles using SLAM and determine an optimal path for the simulated vehicle to follow with the best vehicle dynamics parameters determined by the parameter optimizer. The simulation will also contain a setting to drop a percentage of incoming packets to the digital twin, to simulate the unreliability of the medium in practical applications.

The second domain for the physical twin is in-vehicle, which will use McSCert's Mini M-City one-tenth scale vehicles. The vehicle shall report the same information as the simulated physical twin, with the exception of using a localization technology such as AprilTags to determine the global position of the vehicle in the room in the absence of GPS information.

The digital twin shall run on a server which will receive the periodic information from the digital twin, and periodically solve an optimization problem to update the vehicle dynamics parameters. It shall use a timeseries database to store and retreive previous dynamics information as well. During the parking maneuver, the digital twin shall receive the LiDAR and camera data from the physical twin (either real or simulated), run a path planning algorithm to determine the trajectory to follow, then transmit the path to follow and the vehicle parameters to the physical twin for it to execute the commands to follow the trajectory with its own controller.

2.2 Requirements

Physical Twin Requirements:

- PHYS-1: The physical twin shall be controllable by a human operator using a game controller.
  - PHYS-1.1: The longitudinal motion shall be controllable by the left joystick along the vertical axis.
  - PHYS-1.2: The steering angle shall be controllable by the right joystick along the horizontal axis.
  - PHYS-1.3: To transition to autonomous mode for auto-parking, the "MODE" button on the controller shall be toggled on.
- PHYS-2: The physical twin shall have an auto-park feature.
  - PHYS-2.1: The physical twin shall detect a desired parking pose in the environment by finding a red piece of paper on the ground which is the same size as the vehicle.
  - PHYS-2.2: The physical twin shall transmit LiDAR and camera data to the digital twin to perform obstacle localization.
  - PHYS-2.3: The physical twin shall be able to follow a trajectory provided by the digital twin to within 3cm accuracy.
  - PHYS-2.4: The feature can be overridden by the operator at any point by pressing the "MODE" button on the controller.
- PHYS-4: The physical twin shall transmit IMU/GPS sensor data to the digital twin during manual driving at 20Hz.

Digital Twin Requirements:

- DIGI-1: The digital twin shall periodically save sensor data from the physical twin to a database for further processing at 20Hz.
- DIGI-2: The digital twin shall periodically update its estimate of the vehicle dynamics parameters based on incoming sensor data.

  - PHYS-2.1: The digital twin shall estimate the vehicle's mass, cornering stiffnesses, and yaw inertia to within 10% accuracy.
  - PHYS-2.2: The digital twin shall not perform the parameter update if there is insufficient or inconsistent data from the physical twin, considered anything more than 20% data loss.

- DIGI-3: The digital twin shall be able to localize the poses of obstacles in the environment from the LiDAR and camera data provided by the physical twin.
- DIGI-4: The digital twin shall be able to localize the poses of obstacles in the environment from the LiDAR and camera data provided by the physical twin to within 5cm accuracy.
- DIGI-5: The digital twin shall be able to determine the location of the desired pose by detecting the red paper in the camera frame to within 5cm accuracy.
- DIGI-6: The digital twin shall be able to calculate a trajectory to move the vehicle from its initial pose to the desired final pose.

3 Design and schedule

3.1 Architecture

Figure 1 shows the high-level architecture of the vehicle (physical twin) and the digital twin.

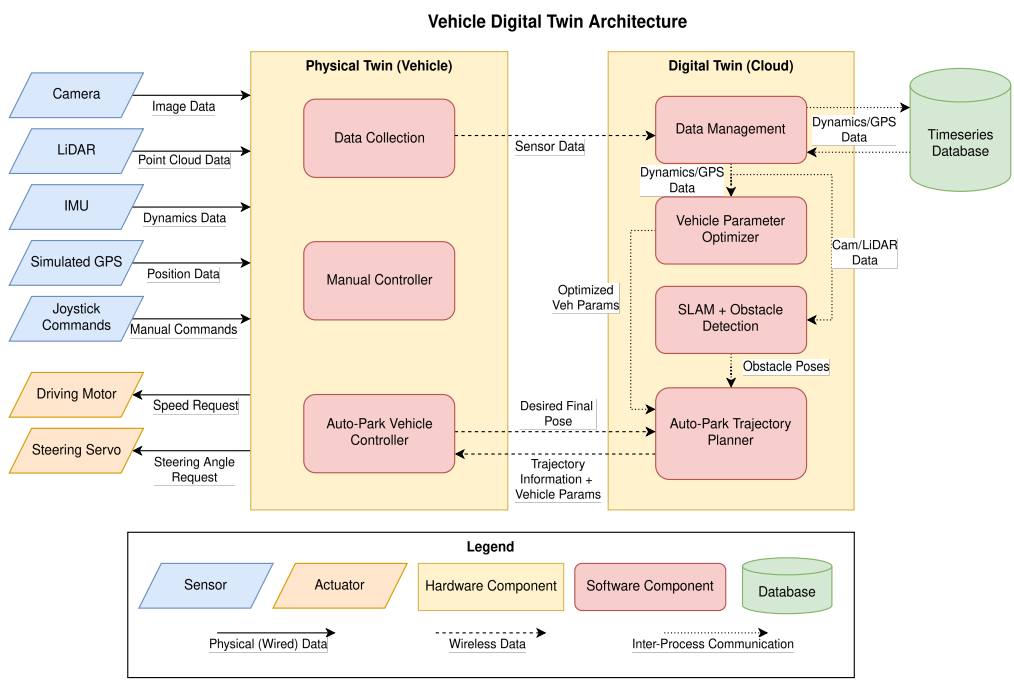

Figure 1: Vehicle Digital Twin Architecture

3.1.1 Hardware

There are two primary hardware platforms which will be used in this project.

- The physical twin vehicle will be running on the RoboRacer one-tenth scale autonomous vehicle stack. It is equipped with a camera, 2D LiDAR, and IMU sensors. The primary compute unit where the vehicle-level software will be deployed is an NVIDIA Jetson Nano.
- The digital twin will be running on a server which can be accessed wirelessly. For the purposes of this project, it will be run locally on my machine, but this could be deployed on a cloud provider as well.

3.1.2 Software

The project is broken up into seven unique software components, three of which run on the physical twin, and four of which run on the digital twin. Each of these software components shall be implemented as a ROS2 node in Python or C++.

The software components on the physical twin are as follows:

- The Data Collection component will take all sensor data from the connected sensors on the vehicle as inputs and is responsible for transmitting it to the Data Management component on the digital twin.
- The Manual Controller will be listening to the connected joystick and actuate the drive/steering motor based on the operator commands.
- The Auto-Park Vehicle Controller will transmit the desired pose to the digital twin and receive the trajectory to follow and updated vehicle dynamics parameters. It will then actuate the driving and steering motors to follow the provided trajectory.

The software components on the digital twin are as follows:

- The Data Management component will receive the sensor data from the physical twin and save them to a timeseries database which can be queried by timestamps to get all data from specific time regions.
- The Vehicle Parameter Optimizer will take the IMU information and the GPS information from the physical twin and periodically run an optimization sweep to determine the most accurate vehicle dynamics parameters. This will be running the dynamic vehicle bicycle model, which requires the vehicle mass, yaw intertia, cornering stiffnesses, and distances from center of gravity to axles.
- The SLAM + Obstacle Detection component will take the LiDAR and camera data from the physical twin and run a SLAM algorithm to localize all the obstacles in the vehicle's environment.
- The Auto-Park Trajectory Planner will take in the obstacle poses from the obstacle detection component and the desired final pose from the physical twin, then send them to the trajectory planner on the digital twin. When it receives the trajectory to follow

and the most up-to-date vehicle dynamics parameters, it will generate the trajectory for the physical twin to follow.

3.1.3 Middleware

All inter-process communication from within each hardware component shall be handled as ROS2 topics. The sensor data during the manual driving phase shall be transmitted as a UDP stream since it is not required for all packets to make it to the server. During an auto-park maneuver, the transmission protocol shall switch to TCP since packet loss is more detrimental in that environment.

3.2 Technology

Each software component will be implemented as a ROS2 node (either C++ or Python), with the inter-process signals using ROS2 topics. If more efficient communication is required for the image or LiDAR data to be transmitted wirelessly, a framework such as ImageZMQ may be considered as well. The timeseries database shall use a timeseries-data oriented extension to PostgreSQL called TimescaleDB.

3.3 Deployment plans

The artifacts for the physcial twin shall be deployed onto the onboard compute unit on the RoboRacer vehicle, the NVIDIA Jetson Orin Nano, and a ROS2 launcher shall manage running all the components together. The components of the digital twin shall run on my laptop and shall also be deployed using the ROS2 launcher.

3.4 Schedule

- Week of Feb 15: Begin implementation of Data Collection, Manual Controller, and Auto-Park Vehicle Controller. Begin setup of simulated physical twin and interfaces between physical twin and digital twin.
- Week of Feb 22: Begin setup of TimescaleDB database, Data Management component, and vehicle parameter optimizer. Perform experiements with simulated vehicle to see if optimizer is able to effectively detect changes in vehicle parameters.
- Week of Mar 1: Begin development of SLAM and obstacle detection algorithm with simulated vehicle. Test with virutalized obstacles.
- Week of Mar 8: Begin development of Auto-Park Trajectory Planner, test end to end auto-park stack in simulation.
- Week of Mar 15: Transition to deploying the software onto the real physical twin in the McSCert lab. Begin final report writeup.
- Mar 22: Submit final report.